ALTINN STUDIO

Harness engineering

Making the work of coding agents something we can trust

Altinn Studio · Digdir

AGENDA

1. Why, and where we are going
2. Model, harness, interface and computer
3. What the agent knows
4. How it checks its work
5. Where it works
6. How we improve it
7. Demo, and where to start

🚀

But first, some harness engineering

Two agents each get a real issue from our backlog, and fix it in a sandbox of their own on a desktop machine. We check on them at the end.

DEMO · START THE AGENTS

Two issues, two agents

agent 1 #19953 a tiny number shown as 3 quadrillion

```
Reproduce and fix https://github.com/Altinn/altinn-studio/issues/19953, with ttd/martiontest as the test app. Have gpt-6.1-sol review the change in fast mode, then open a draft pull request with a recording of before and after.
```

agent 2 #21584 image builds that send too much

```
Fix https://github.com/Altinn/altinn-studio/issues/21584 for the images that build from src/. Open a draft pull request with a table of build time and context size before and after.
```

That is all they get.

The rest is the harness: what they know, the checks they run and the computer they run them on.

WHERE WE ARE

More to deliver, with fewer people



Altinn 3 is done

The migration is finished. What it left behind is still ours to clean up.



Lots of technical debt

Code, infrastructure and docs that need work, alongside everything new.



People are leaving

Consultants with valuable knowledge are leaving, and what they know leaves with them.



Still lots to deliver

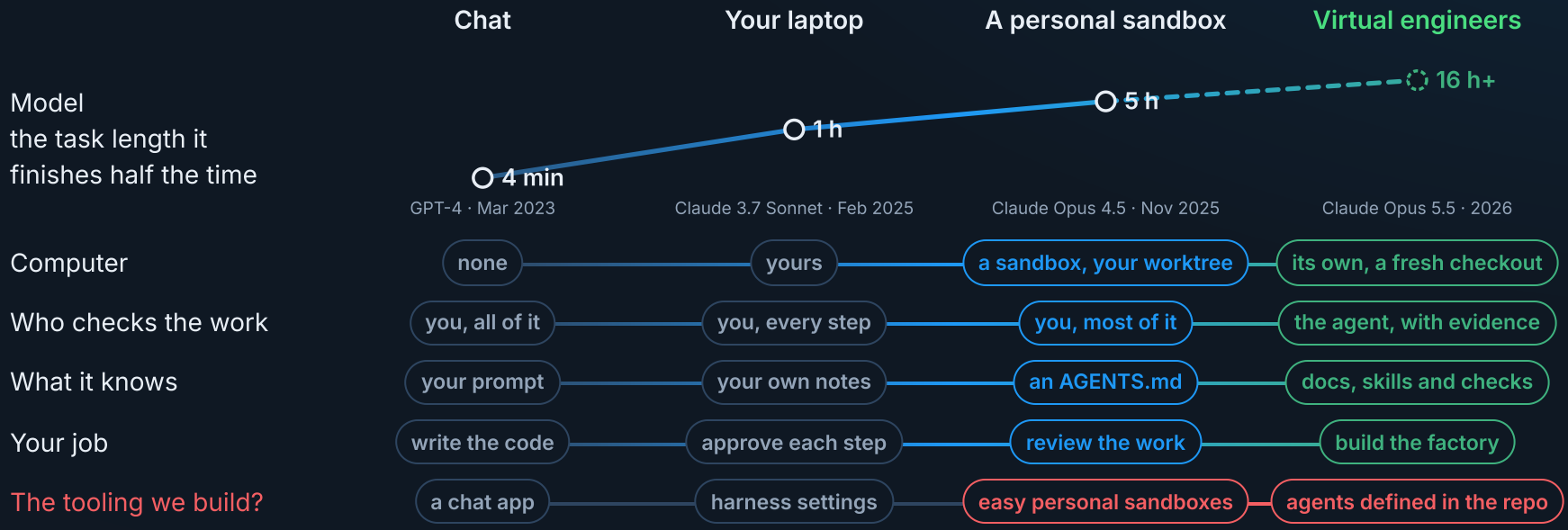
Service owners and the roadmap don't wait for us to catch up.

We need **more done per person**, without **lowering quality**.
That is why we are going deep on AI-native engineering.

"Coding agents now write much of our code, and they need the same knowledge about our systems as the people on the team."
Altinn Studio's documentation ADR, October 2026

WHERE WE ARE GOING

From a chat window to virtual engineers



Models like Opus 5.5 can already do the work on the right.
What holds us back is the **harness**, and our **ambition**.

TERMS

Model, harness, interface

model

Generates text

Claude, GPT, Qwen. On its own it can only write. It cannot read a file or run a test.

harness

Lets the model act

Tools such as read, edit, bash and search, the loop that runs them and feeds back the results, and the instructions and skills it loads. Claude Code, Codex, Pi.

interface

Where people come in

A terminal, an editor or a desktop app. Claude Code ships one with its harness, but the two are separate.



The model is the **chef**. The harness is the **kitchen**, with its tools. A tool like T3 Code is the **restaurant**, which lets you bring whichever kitchen you like.

After Theo Browne, "What is a 'harness'?", October 2026

OUR ADDITION

An agent also needs a computer

agent = computer + model + harness

Your laptop — shared

- **Your credentials**
  tokens, SSH keys, cloud logins and the company network
- **Permission prompts**
  approve every command, or trust it with your machine
- **One at a time**
  one set of ports, one Docker, one localtest

Its own computer — agentctl sandbox

- **No real secrets inside**
  the host adds them only in requests to their allowed hosts
- **Root and no prompts**
  the blast radius is one virtual machine
- **As many as you can review**
  each with its own checkout, cluster and browser

We stopped sharing our computers with agents for **security**, and it made them **more productive**.

DEFINITION

Harness engineering is designing everything around the model, so that we can trust what comes out.

What it knows
docs, instructions, skills

How it checks its work
tests, checks, running it for real

Where it works
its computer and its tools

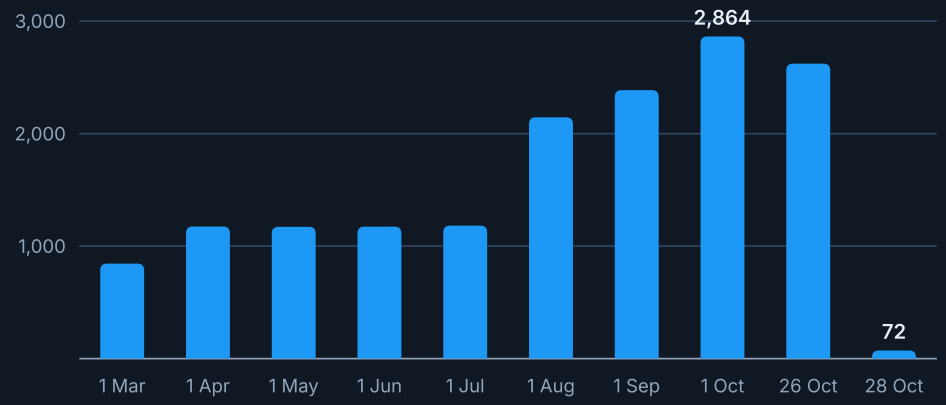
How we improve it
every mistake fixes the harness

The term comes from OpenAI and Thoughtworks, early 2026: engineers design the environment and the feedback loops, and agents write the code.

WHAT IT KNOWS

Instructions that nobody reviewed

Lines in all `AGENTS.md` files in altinn-studio, 2026



39 → 10 AGENTS.md files

2,864 → 72 lines

Generated, not reviewed

Agents wrote much of it as they worked, and people rarely read it, so it drifted from the code and the READMEs.

Written for older models

Much of it worked around what early 2026 models got wrong, or described code that today's models read for themselves.

Not really about agents

Most of it was good documentation for anyone. It moved into READMEs and docs/, where people read and review it too.

WHAT IT KNOWS

Write for people, and agents read it too

README, docs/, CONTRIBUTING — people and agents
what a component is for, how we work, why we decided

AGENTS.md — agents, every task
only what is specific to agents, in a few lines

Skills — agents, some tasks
procedures loaded on demand: app development, changelog, PR evidence, plain Norwegian

Agent definition — one computer
how this agent works, and what's special about its computer

Agent-specific, from our root AGENTS.md

"What you learn about the product or how we work goes into the repository through a pull request, not into your harness's memory, so that the rest of the team can read and review it."

Written for people first

What a person also needs goes in the docs everyone reads and reviews.

Plain Markdown for any harness

No CLAUDE.md and no harness-specific imports. CI checks both.

HOW IT CHECKS ITS WORK

Verification is what lets trust grow

Builds, formats, lints	seconds
Unit and integration tests	minutes
Repository checks: docs, spelling, changelog, PR title	CI
Runs the system: Studio, localtest, kind	own computer
Looks at the result: browser, desktop, Cypress	own computer
Shows a person: screenshots and recordings	review
Users try it, and their feedback sets the next step	we steer

- Every check an agent runs itself is one **you don't have to do**
- The further down the list, the closer it gets to **what users see**
- The bottom half needs **a full computer of its own**: containers, clusters, a browser and root
- On **our own laptops** we have to hold an agent back, and soon policy won't allow it there at all
- The last check is **the users**: we listen, and we decide where to go next
- So **quality rises together with productivity**

HOW IT CHECKS ITS WORK

Checks that say what to do next

```
Documentation validation failed with 1 problem(s):

  - src/x/CLAUDE.md: CLAUDE.md is no longer used — put the
    guidance in src/x/AGENTS.md, which every agent reads
    directly

See docs/documentation.md.
```

From `yarn docs:validate`, which CI runs on every pull request. An agent reads the failure and fixes it without asking. The agent image has the same tools as CI, so it runs the same checks before it pushes.

Script what can be scripted

What agents do often, or often get wrong, becomes a script or a CI check. Their attention and context go to the parts that need judgment.

Cheap to build now

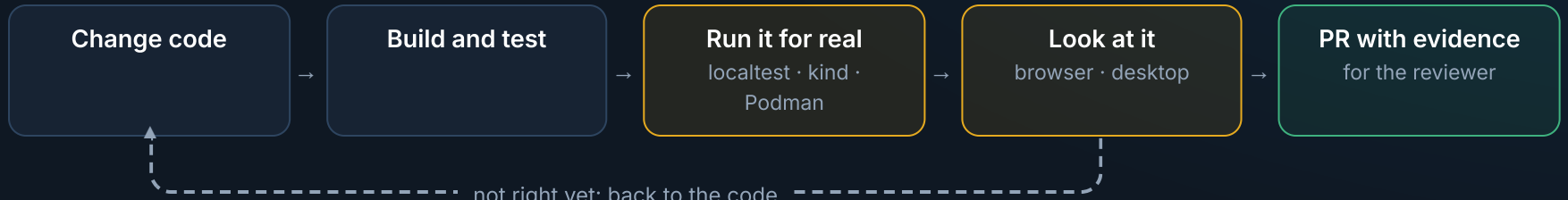
An agent writes the check, and proves that it catches the problem, in minutes.

Read by people and agents

The same message tells a person and an agent what to fix.

HOW IT CHECKS ITS WORK

Run it, look at it, show it



The whole stack

`studioctl` runs Studio, localtest and apps. kind runs the cluster parts.

A desktop it can drive

Chromium, Playwright and a computer-use skill, so it sees what a user would see.

Evidence for the reviewer

The pr-evidence skill attaches screenshots and recordings from the tested revision, so you review the result and not only the diff.

WHERE IT WORKS

The computer is part of the harness

```
altinn-studio/agents/
├─ Dockerfile · every toolchain the repo needs
├─ smoke.sh · proves it, in CI
├─ instructions.md · role and habits
├─ _skills/ · one home for skills
├─ minimal/
└─ desktop/
   ├─ agent.yaml · resources, secrets, network
   ├─ agent.nested.yaml
   └─ environment.md · what's special about it
```

Ready to verify

.NET 8, 10 and 11, Go, Rust, Node, Python, Podman, kind, Chromium and studioctl, logged in to Studio at boot.

Checks its own computer

An agent that changes the image boots it in a nested agent and builds and tests the repository there, before anyone else gets it, as in #21492.

Reviewed like code

A better toolbox or skill reaches everyone's agents on their next apply.

WHERE IT WORKS

Swap any layer, keep what's ours

Interface

TUI · T3 Code · VS Code · desktop apps

any client, over SSH

Harness

Claude Code · Codex · Pi

an adapter for each

Model

Claude · GPT · open models on aivar

one line in the manifest

Computer

OCI images · Linux, macOS, Windows · libkrun microVMs

any image, in a microVM

▲ ▲ ▲ ▲

agentctl and the Sandbox SDK

make each layer above swappable · built centrally, for all of Digdir

Altinn Studio

specific to the product and its domain · built by the team

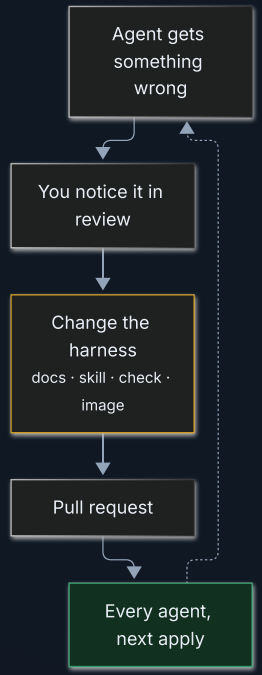
What it knows
docs, AGENTS.md and skills

How it checks its work
tests, CI checks and scripts

Agent definitions
the image, manifest and instructions

HOW WE IMPROVE IT

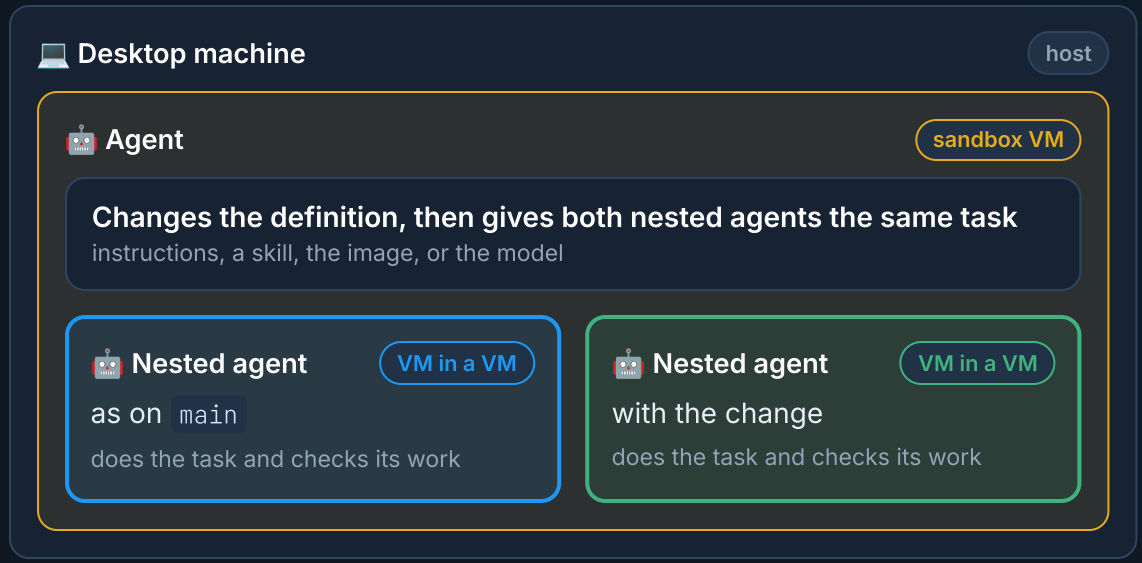
Fix the harness, not the output



- **Don't add AI attribution to commits** · #20457
- **Run on Norwegian time** · #20465
- **Make Chromium trust the mediated network** · #20537
- **Write clickable links and resolvable file paths** · #20990
- **Record local development gotchas** · #21056
- **A skill for dependency alerts** · #21234

77 changes to `agents/` since it was added on 26 August

Test a change in an agent inside the agent



→

A comparison
which checks passed, what each delivered, time and cost

A pull request
with the result as evidence

Done for real

#21492 added the toolchains the repository needs, and proved them in a nested agent built from the branch.

Next: evaluations

One task, two nested agents, one difference, to measure whether a change to the harness helps.

DEMO

Back to the two agents

agent 1

The tiny number

How it reproduced #19953 in a browser, the fix and its test, the review, and the recording before and after.

agent 2

The image builds

What it changed for #21584, and its table of build time and context size before and after.

HONEST LIMITS

What we haven’t solved

Review is the bottleneck

Agents open pull requests faster than people review them. Better evidence helps, but a person still decides.

Behaviour is hardest to check

Tests written by the same agent that wrote the code can share its mistakes. Running the real thing is our best answer so far.

We don't measure the harness yet

We only run evaluations by hand, now and then. We don't yet know which lines of a skill make a difference.

It needs upkeep

Harness versions, images and docs change every week, and memory still piles up in personal harnesses.

WHERE TO START

One step to the right

Chat — Your laptop — A personal sandbox — Virtual engineers

If you are here, next →

Do real work

Use a harness such as Claude Code or Codex on real tasks in your own repository.

If you are here, next →

Give it a computer

Move the agent into a sandbox of its own, and stop approving every command.

If you are here, next →

Into the repository

Check in the agent definition, so it starts from a fresh checkout and runs its own checks.

If you are here, next →

Measure and improve

Evaluate the harness, and fix it through pull requests whenever an agent gets something wrong.

Let's discuss it in **#team-agentic-sdlc**.

THANKS

Questions?

Join us on Slack in **#team-agentic-sdlc**

where we develop agentctl and talk about how we work with agents

github.com/Altinn/altinn-studio · `agents/` · `docs/documentation.md`

github.com/digdir/digdir-agents · `agentctl/README.md` · `agents/examples/`